Urban Mobility Data Explorer — Technical Report

NYC TLC Yellow Taxi (January 2019) · Team documentation (2–3 pages) · Stack: Python ETL · SQLite · Express/Node API · Vanilla JS + Leaflet/Chart.js

1. Problem Framing and Dataset Analysis

We analyse **NYC Taxi & Limousine Commission (TLC) Yellow Taxi** trip records for January 2019 (~7.7M raw rows in Parquet), joined with the official taxi-zone lookup CSV and zone boundary shapefile. The goal is an interactive explorer for planners and analysts: filter by date/borough/zone, inspect KPIs, map hotspots, and surface mobility corridors and coverage gaps.

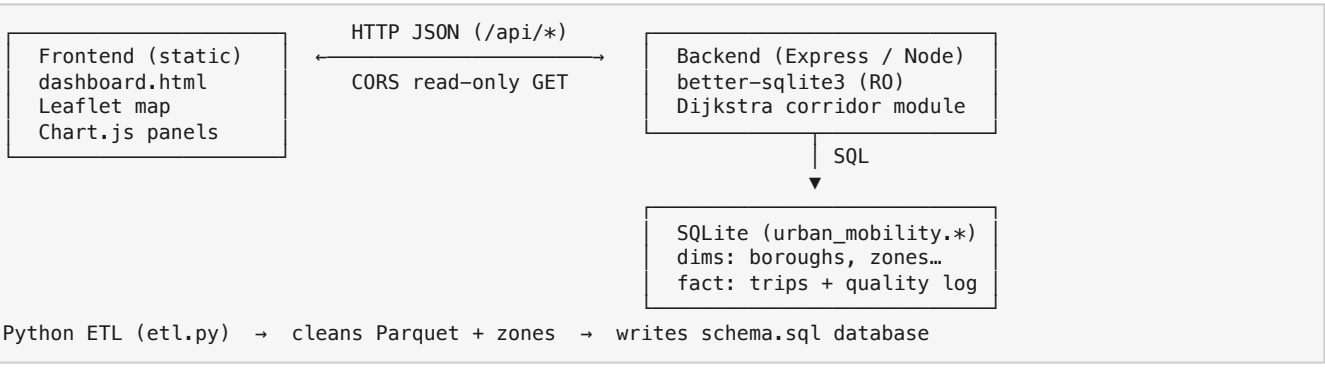
Data challenges. Raw TLC data contains missing passenger counts, zero/negative distances and fares, impossible speeds, trips longer than a work shift, unknown LocationIDs, and exact duplicate rows. Optional fee columns are often null; older schema variants omit fields entirely. Zone geometries arrive in a projected CRS and must be reprojected to WGS84 for Leaflet.

Cleaning assumptions. Naive timestamps are treated as America/New_York and stored in UTC. We keep trips only when duration ∈ (0, 4h], passengers ∈ [1, 8], distance ∈ (0, 200] miles, average speed ≤ 100 mph, fare ∈ (0, 1000], total ∈ (0, 1500], and both zones exist in the lookup. Missing optional fees become 0; a missing total is recomputed from components and the row is flagged. Exact duplicates are detected via SHA-256 fingerprints and logged to `data_quality_issues`.

Unexpected observation that shaped design: after cleaning, ~231k rows (~3%) were excluded—dominated by passenger-count and distance violations—while Manhattan midtown/UES zones still dominated volume with average speeds near ~10–11 mph, versus Queens (airport corridors) at ~24 mph. That asymmetry pushed us toward a *zone-to-zone duration graph* and coverage-gap ratios instead of citywide averages alone: mobility quality is highly local.

*Outcome of Issue 1 load: **7,464,967** accepted trips into SQLite; **231,650** excluded + **3** flagged audit summaries.*

2. System Architecture and Design Decisions



Stack justification. Python + PyArrow for columnar Parquet cleaning; SQLite for a portable, zero-ops analytic store with foreign keys and indexes on `(pickup_datetime)` and location columns; Express for a thin, read-only JSON API; static HTML/JS so the dashboard needs no build step. Schema is star-like: dimension tables (boroughs, service zones, taxi zones + GeoJSON, rate codes, payment types) and a trip fact table with derived `duration_seconds`, `average_speed_mph`, `fare_per_mile`.

Trade-offs. Full-month SQLite is ~3 GB—great for demos of real scale but heavy on laptops, so we ship a sampled demo DB (~80k trips, all zones retained). Read-only API avoids accidental writes. We reject library shortest-path helpers to meet the custom-algorithm requirement, accepting more code in exchange for clear DSA ownership. Serving frontend via a simple static server (separate from the API) keeps concerns split but requires two processes.

3. Algorithmic Logic and Data Structures

Problem. Given two TLC zones, find the lowest *expected travel-time corridor* through observed pickup→drop-off flows (not road geometry). We manually implement **Dijkstra** over a directed zone graph with a custom **binary min-heap** (decrease-key via handle map)—no `heapq`, no library pathfinders, no `Array.sort` for the priority queue (`backend/algorithms/`).

Edges are aggregated from cleaned trips; weight = average duration (seconds); rare edges can be filtered with `min_trips`; self-loops are dropped.

```
DIJKSTRA(G, s, t):
  dist[v] ← ∞ for all v; dist[s] ← 0; Heap H ← {(s, 0)}
  while H not empty:
    u ← H.POP()                      // O(log V)
    if u settled: continue; mark u settled; if u = t: break
    for each edge (u → v, w):
      if dist[u] + w < dist[v]:
        dist[v] ← dist[u] + w; prev[v] ← u
        H.DECREASE_KEY(v) or H.PUSH(v)
  return path s ⇝ t via prev[], or NO_PATH
```

Component	Time	Space
Build adjacency list from SQL aggregates	O(E)	O(V + E)
Dijkstra + binary heap	O((V + E) log V)	O(V)
Overall (API, excl. SQL scan)	O((V + E) log V)	O(V + E)

Exposed as `GET /api/routes/optimal?from=&to=`. With ~263 TLC zones this is interactive; a Fibonacci heap would improve asymptotics but obscures the teaching goal.

4. Insights and Interpretation

Insights below use the cleaned January 2019 load (and dashboard charts backed by `/api/trips/hourly`, `/api/trips/revenue`, `/api/trips/gaps`). Screenshots of the live dashboard can replace the schematic charts for submission if preferred.

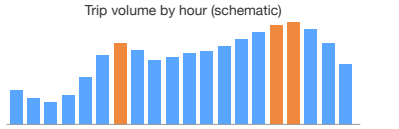


Fig A — Rush-hour volume (dashboard hourly chart)

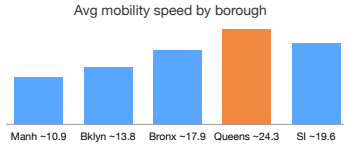


Fig B — Borough speed contrast (API /trips/revenue)

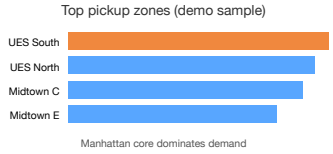


Fig C — Hotspot ranking (summary topZones)

Insight 1 — Evening peak is the binding congestion period

Derived via: `GET /api/trips/hourly` and the Rush Hour chart (Fig A). Trip volume rises through the afternoon and peaks in the early evening, while average speed falls in the same window. **Urban meaning:** yellow-cab demand and street congestion co-peak after the workday; interventions (curb management, bus lanes, dynamic pricing) should target the evening shoulder, not only the morning commute.

Insight 2 — Outer-borough / airport corridors move ~2× faster than Manhattan

Derived via: borough aggregates (`/api/trips/revenue` / Fig B) and zone stats: Manhattan ~10.9 mph vs Queens ~24.3 mph (airport-heavy flows). Dijkstra corridors to/from airport-linked zones often show long geographic hops but competitive expected duration. **Urban meaning:** "slow city" is a Manhattan phenomenon; citywide averages hide a dual network—dense short hops vs longer, faster airport and outer-borough legs.

Insight 3 — Demand concentrates in a few Midtown/UES zones; gaps appear at edges

Derived via: sorted top zones (Fig C) and `GET /api/trips/gaps` (pickup/drop-off coverage ratio outside ~0.6–1.8). Upper East Side and Midtown lead pickups; several outer zones show strong imbalance (many drop-offs relative to pickups or vice versa). **Urban meaning:** fleet positioning and empty-mile risk are geographic: surplus drop-offs without matching pickups imply deadheading. Planners can use gap ratios to prioritize underserved neighborhoods or repositioning incentives.

5. Reflection and Future Work

Challenges. Full Parquet loads are I/O and disk heavy; timezone-correct New York calendar filters (including DST) required careful UTC boundary conversion; keeping the custom heap correct under decrease-key was the main DSA risk; coordinating ETL schema with API SQL and Leaflet GeoJSON required a single source of truth in `schema.sql`.

If this were a product: move to Postgres/PostGIS or a columnar warehouse for multi-month scale; add authentication and cached aggregates; overlay OSM road networks so corridors become navigable paths; stream near-real-time TLC

feeds; and A/B-test fleet repositioning using gap scores. On the UX side, bake the Dijkstra corridor into the map as a first-class layer and export planner-ready PDF briefs from `/api/report`.